



UNIVERSIDADE FEDERAL DE ALAGOAS - UFAL

INSTITUTO DE COMPUTAÇÃO

Relatório de Projeto Acadêmico

ROMA QUIZ: jogo em formato de quiz educacional com arquitetura de microserviços.

Quiz online sobre o Império Romano com frontend Next.js, microserviços Express, inteligência artificial e banco PostgreSQL.

Alunos: Caio Mascarenhas Soares

Link do GitHub: <https://github.com/CaioMascarenhas/quiz-imperio-romano>

Link do vídeo demonstrativo: <https://youtu.be/sXvT0QiAKIM>

Maceió - AL
2026

SUMÁRIO

1. Enunciado	3
2. Product Backlog	3
2.1. Sprint 1 - Infraestrutura Base	3
2.2. Sprint 2 - Lógica do Quiz	4
2.3. Sprint 3 - Interface e Experiência	4
3. Sprint Backlog	4
4. Arquitetura do Sistema	5
5. Diagrama de Componentes	6
6. Tecnologias Utilizadas	7
7. Microserviços Implementados	8
8. Funcionamento do Sistema	10
9. API Endpoints	12
10. Testes e Validação	13
11. Links do Projeto	14
12. Considerações Finais	14
13. Conclusão	15

1. ENUNCIADO

O projeto Roma Quiz consiste em uma aplicação web educacional voltada ao estudo do Império Romano. A proposta é oferecer ao usuário uma experiência interativa de aprendizagem por meio de quizzes temáticos, chat com inteligência artificial, ranking de pontuação e histórico de respostas.

A aplicação foi desenvolvida com uma arquitetura distribuída baseada em microserviços. O frontend é responsável pela experiência visual e comunicação com as APIs, enquanto os serviços de backend cuidam separadamente da autenticação, geração de questões, chat, ranking e persistência de tentativas.

O sistema utiliza Docker Compose para orquestrar os serviços e PostgreSQL como banco de dados compartilhado. A inteligência artificial é usada no serviço de quiz para gerar perguntas educativas e respostas de chat sobre o Império Romano.

2. PRODUCT BACKLOG

Item	Descrição	Prioridade
Cadastro e login	Permitir criação de conta, autenticação do usuário e emissão de token JWT.	Alta
Quiz por tema	Gerar perguntas sobre temas do Império Romano usando IA e retornar alternativas, resposta correta e explicações.	Alta
Histórico	Salvar tentativas respondidas pelo usuário e exibir respostas escolhidas, corretas e explicações.	Alta
Ranking	Exibir usuários ordenados por quantidade de acertos, com títulos de acordo com a colocação.	Média
Chat educativo	Permitir perguntas abertas sobre o Império Romano usando IA com contexto recente da conversa.	Média
Dockerização	Executar frontend, backends e banco por meio de Docker Compose.	Alta

2.1. Sprint 1 - Infraestrutura Base

A primeira sprint concentrou-se na criação da estrutura do projeto, separando frontend, backends e banco de dados. Também foram definidos os containers, as portas de execução e as variáveis de ambiente necessárias para o funcionamento local.

2.2. Sprint 2 - Lógica do Quiz

A segunda sprint implementou a integração com a IA para geração das questões, o formato JSON retornado pelo serviço de quiz e a gravação de tentativas no banco de dados. Também foram criadas as tabelas de usuários e respostas.

2.3. Sprint 3 - Interface e Experiência

A terceira sprint concentrou-se na interface Next.js, com páginas de landing page, login, cadastro, dashboard, quiz, chat, ranking e histórico. Foram adicionadas imagens relacionadas ao Império Romano e animação Lottie durante a geração das questões.

3. SPRINT BACKLOG

Sprint	Tarefas	Status
Sprint 1	Configurar Docker Compose, PostgreSQL, estrutura dos microserviços e rotas de saúde.	Concluído
Sprint 2	Implementar autenticação JWT, geração de perguntas com IA e persistência das tentativas.	Concluído
Sprint 3	Construir telas do usuário, ranking, histórico, chat e integração do frontend com as APIs.	Concluído

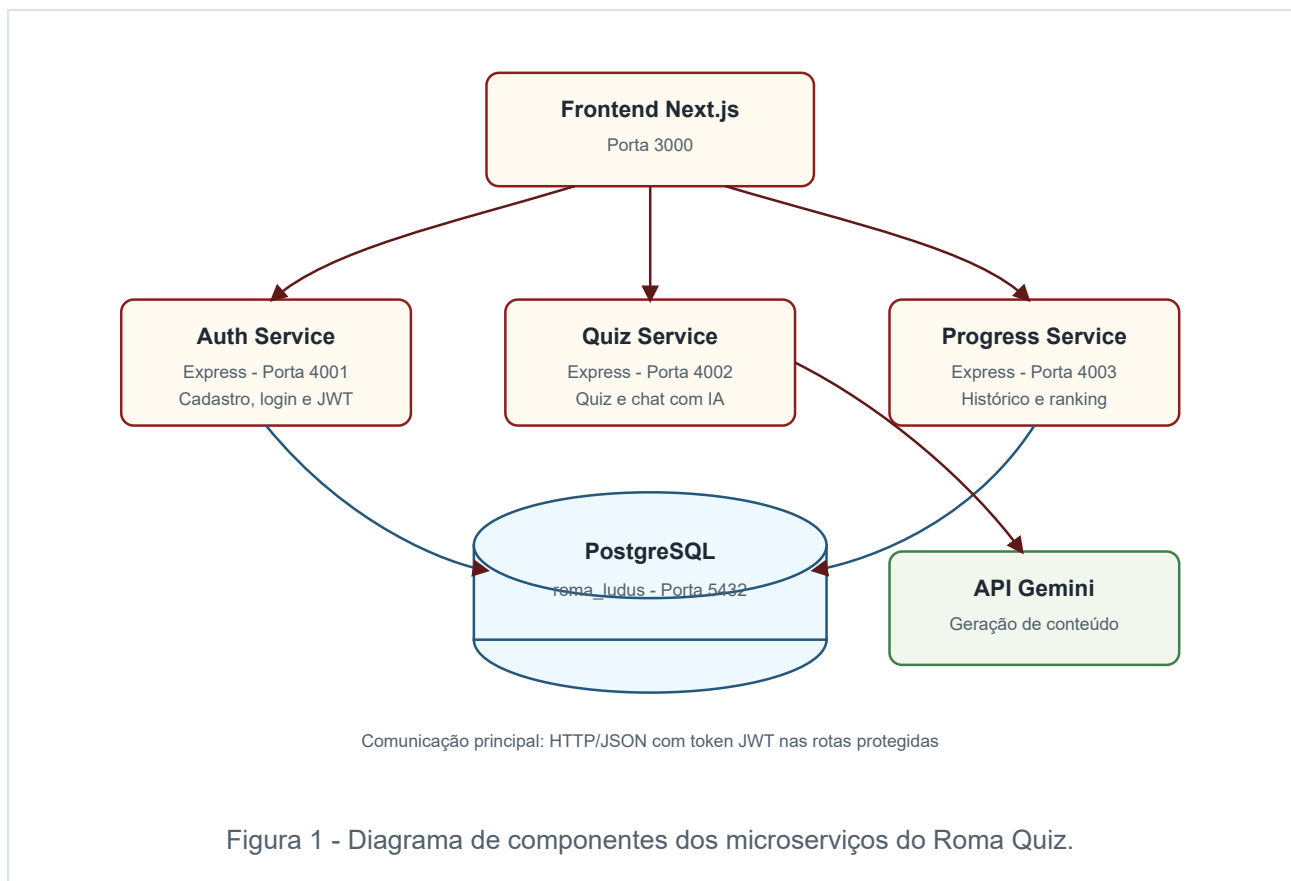
4. ARQUITETURA DO SISTEMA

A arquitetura do Roma Quiz é organizada em serviços independentes. O frontend Next.js consome os microserviços por HTTP. O `auth-service` gerencia cadastro, login e validação de sessão. O `quiz-service` centraliza a geração de perguntas e respostas de chat por meio da API Gemini. O `progress-service` registra tentativas, consulta histórico e calcula o ranking.

O banco PostgreSQL é compartilhado entre os serviços que precisam de persistência. Essa decisão simplifica a execução acadêmica do projeto, mantendo a separação lógica dos domínios por serviço.

Características da arquitetura: separação por responsabilidade, comunicação HTTP/JSON, autenticação por JWT, persistência em PostgreSQL, orquestração com Docker Compose e integração com IA para conteúdo educacional.

5. DIAGRAMA DE COMPONENTES



6. TECNOLOGIAS UTILIZADAS

Backend

Os microserviços foram implementados com Node.js, Express, CORS, Dotenv, JSON Web Token e PostgreSQL. O serviço de autenticação utiliza BcryptJS para hash de senhas. O serviço de quiz utiliza `@google/genai` para integração com a API Gemini.

Frontend

O frontend utiliza Next.js 16, React 19, TypeScript, Tailwind CSS, Radix UI, componentes reutilizáveis, Lucide React, Lottie React e bibliotecas auxiliares para construção da interface.

DevOps

O ambiente é executado por Docker Compose, com containers separados para frontend, autenticação, quiz, progresso e PostgreSQL.

Arquitetura

A aplicação segue uma arquitetura orientada a microserviços, com comunicação HTTP, payloads JSON, autenticação por token e persistência relacional.

7. MICROSERVIÇOS IMPLEMENTADOS

7.1. Frontend Next.js - Porta 3000

Responsável pela interface do usuário. Inclui landing page, páginas de login e cadastro, dashboard, tela de quiz, chat, ranking e histórico. O frontend armazena o token no navegador e chama os serviços por meio das URLs públicas configuradas no Docker Compose.

7.2. Auth Service - Porta 4001

Responsável por cadastro, login, hash de senha e geração de token JWT. Também possui rota para validar a sessão atual do usuário. O serviço cria a tabela `users` no PostgreSQL quando iniciado.

7.3. Quiz Service - Porta 4002

Responsável por gerar quizzes e responder ao chat educacional. O serviço valida o JWT nas rotas protegidas e consulta a API Gemini para produzir perguntas, alternativas, feedbacks e explicações sobre o Império Romano.

7.4. Progress Service - Porta 4003

Responsável por registrar tentativas de quiz, consultar o histórico individual e gerar ranking dos usuários. O serviço cria a tabela `quiz_attempts` e relaciona cada tentativa ao usuário autenticado.

7.5. PostgreSQL - Porta 5432

Banco de dados relacional utilizado para persistir usuários e tentativas. A base configurada no Docker Compose chama-se `roma_ludus`.

8. FUNCIONAMENTO DO SISTEMA

8.1. Fluxo de Autenticação

O usuário cria uma conta informando nome, e-mail e senha. O `auth-service` aplica hash na senha, grava o usuário no banco e retorna um token JWT. Esse token é enviado nas chamadas protegidas para quiz, chat, ranking e histórico.

8.2. Tela de Login e Registro

As telas de login e registro fazem a comunicação com o serviço de autenticação. Após o sucesso, o usuário é direcionado para a área principal da aplicação.

8.3. Geração de Quiz

O usuário escolhe um tema relacionado ao Império Romano. O frontend envia o tema para `/quiz/generate`. O serviço monta um prompt, solicita a geração à IA e retorna cinco questões em JSON, cada uma com enunciado, alternativas, resposta correta, explicação e feedbacks.

8.4. Registro de Respostas

Ao responder uma questão, o frontend envia a tentativa ao `progress-service`. A tentativa fica associada ao usuário autenticado, permitindo montar histórico e ranking.

8.5. Chat Educativo

O chat usa o `quiz-service` para responder perguntas em português do Brasil. O serviço envia até cinco mensagens recentes como contexto, mantendo continuidade na conversa.

8.6. Ranking e Histórico

O ranking ordena usuários por acertos, enquanto o histórico lista as tentativas do usuário com tema, pergunta, resposta escolhida, resposta correta e explicação.

9. API ENDPOINTS

Serviço	Endpoint	Método	Finalidade
Auth	/health	GET	Verifica se o serviço está ativo.
Auth	/register	POST	Cadastra usuário e retorna token JWT.
Auth	/login	POST	Autentica usuário existente.
Auth	/me	GET	Valida o token e retorna dados do usuário.
Quiz	/quiz/generate	POST	Gera questões por tema usando IA.
Quiz	/chat	POST	Responde perguntas educacionais sobre o Império Romano.
Progress	/attempts	POST	Registra tentativa respondida pelo usuário.
Progress	/ranking	GET	Consulta ranking geral ou por tema.
Progress	/history	GET	Consulta histórico do usuário autenticado.

10. TESTES E VALIDAÇÃO

A validação do projeto pode ser realizada com execução local via Docker Compose e testes manuais das principais funcionalidades. Também é possível validar as rotas de saúde de cada microserviço antes de testar os fluxos completos.

10.1. Testes Manuais Realizados

- Criação de conta com nome, e-mail e senha.
- Login e armazenamento do token de sessão.
- Geração de quiz por tema com cinco questões.
- Envio de pergunta ao chat educativo.
- Registro de tentativa após resposta do usuário.
- Consulta de ranking e histórico.

10.2. Comandos de Validação

Para executar o projeto completo:

```
docker compose up --build
```

Após a inicialização, acessar `http://localhost:3000`. As APIs ficam disponíveis em `4001`, `4002` e `4003`.

11. LINKS DO PROJETO

Repositório GitHub do projeto:

```
https://github.com/CaioMascarenhas/quiz-imperio-romano
```

Vídeo demonstrativo no YouTube:

```
https://youtu.be/sXvT0QiAKIM
```

12. CONSIDERAÇÕES FINAIS

12.1. Objetivos Alcançados

O projeto alcançou o objetivo de construir uma aplicação educacional interativa sobre o Império Romano, com autenticação, geração de quiz por IA, chat, ranking, histórico e execução em containers.

12.2. Aprendizados

O desenvolvimento permitiu aplicar conceitos de microserviços, APIs REST, autenticação com JWT, persistência relacional, integração com IA generativa e construção de interfaces modernas com Next.js.

12.3. Melhorias Futuras

- Adicionar testes automatizados de integração para as APIs.
- Separar bancos de dados por microserviço para maior independência.
- Persistir conversas do chat para consulta posterior.
- Adicionar painel administrativo para curadoria de temas e imagens.
- Implementar observabilidade com logs estruturados e métricas.

12.4. Desafios Enfrentados

Entre os principais desafios estão a padronização do JSON gerado pela IA, a validação de sessão entre serviços independentes e a organização do ambiente Docker para que frontend, backends e banco funcionem de forma integrada.

13. CONCLUSÃO

O Roma Quiz demonstra a aplicação prática de uma arquitetura baseada em microserviços em um contexto educacional. A separação entre autenticação, quiz, progresso e interface torna o sistema mais organizado e facilita a manutenção. O uso de IA amplia a dinâmica do aprendizado ao permitir geração de questões e respostas contextualizadas sobre o Império Romano.

Assim, o projeto atende aos requisitos de uma aplicação web completa, com interface, backend, persistência, autenticação, componentes distribuídos e documentação arquitetural.